ShelfTwin

Real-time shelf auditing with trustworthy restock and displacement alerts

Track 1: Shelf Auditing and Real-Time Stockouts. Submission: idea and prototype.

At a glance	
Problem	About 1 in 12 products is missing from the shelf at any moment. Most gaps start inside the store and are found too late.
Solution	A per-slot shelf record ('twin') built from cameras. A temporal engine decides when to act, wait, or report that it cannot see.
Techniques	Dense detection (YOLO), catalogue retrieval (DINOv2 + FAISS), OCR tie-breaks, planogram alignment, Bayesian evidence over frames.
Prototype	Working prototype; code and trained models are public. In simulation: 0.12 false alerts per camera-hour vs 966 for a single-frame detector, at full recall.
Deployment	One edge GPU box per store; existing CCTV, shelf cameras or phones; alerts on web, Telegram or WhatsApp.

Team ShelfTwin, SVNIT Surat | github.com/Purv-Kabaria/shelftwin



Problem statement and requirements

PROBLEM STATEMENT (TRACK 1)

"Design and build an automated computer vision solution that processes real-time imagery from shelf-scanning cameras, compares it against a digital planogram, and automatically triggers immediate re-stock or displacement alerts the moment inventory thresholds drop or items are misplaced."

Context given by the organisers

- Human-led shelf audits are slow and error-prone.
- They cannot catch stockouts or misplaced items in real time.
- Smart camera networks or roaming robots scan aisles continuously.
- The scan is compared with the store planogram to raise alerts for backroom staff.

#	Requirement	What it means in practice	How ShelfTwin meets it
R1	Real-time imagery	Read shelf cameras, CCTV or phone video continuously	Edge pipeline at 0.5-2 frames/s per camera; change-triggered analysis
R2	Planogram comparison	Know which product belongs in each slot	Approved layout; per-row sequence alignment of detections to slots
R3	Threshold alerts	Warn when a slot runs low, not only when empty	Per-slot coverage states; adaptive low-stock threshold; time-to-empty
R4	Displacement alerts	Detect a product in the wrong slot	Catalogue match with abstention; identity kept separate from location
R5	Immediate staff alerts	The right person gets a clear task fast	Ranked tasks with photos, stock location, reason, and camera verification
R6	Trust (our addition)	False alarms waste time and teach staff to ignore alerts	Temporal evidence engine with an explicit 'cannot see' state

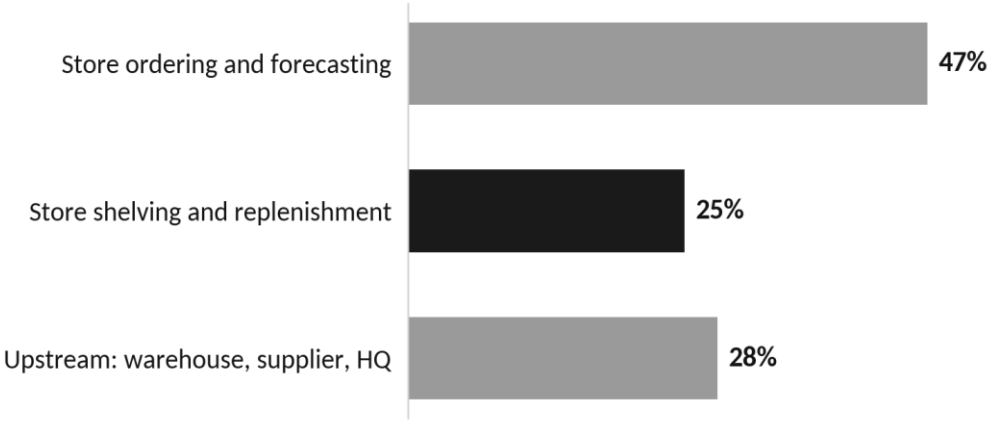
The facts: shelf gaps are frequent, costly, and mostly caused in-store

Figure	What it means
8.3%	Average out-of-stock rate across 661 outlets in a worldwide study
Above 10%	Out-of-stock rate for fast-moving and promoted items, which earn the most
70-75%	Share of out-of-stocks caused by store practice: ordering, forecasting, shelving
About 25%	Cases where the product is in the store but not on the shelf; a camera sees this
31% / 9%	Shoppers who buy at another store / do not buy at all when the item is missing
Up to 4%	Share of yearly revenue lost by grocery retailers to out-of-stocks
Daily or weekly	Typical manual audit cycle; a fast mover can empty within an hour

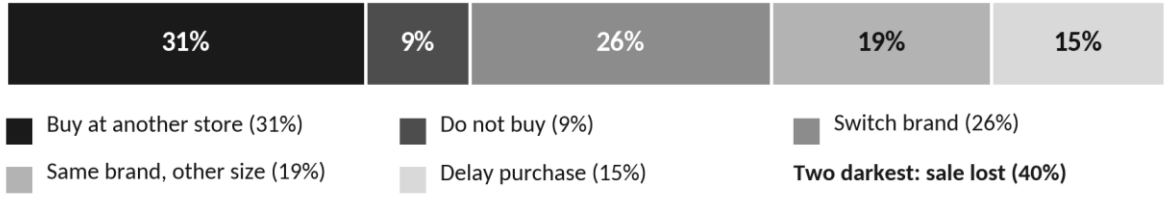
Why this is harder in Indian retail

- Few stores keep a maintained digital planogram.
- Frequent pack redesigns, regional variants and look-alike multipacks.
- Patchy internet; tight hardware budgets; high staff turnover.

Root causes of out-of-stocks (share of cases)



What shoppers do when the item is missing



Both charts: worldwide averages across 661 outlets. Responses vary by category.

Main implication: the store can fix most gaps itself, if it learns about them early and trusts the signal.

Problem breakdown: pain points and our response

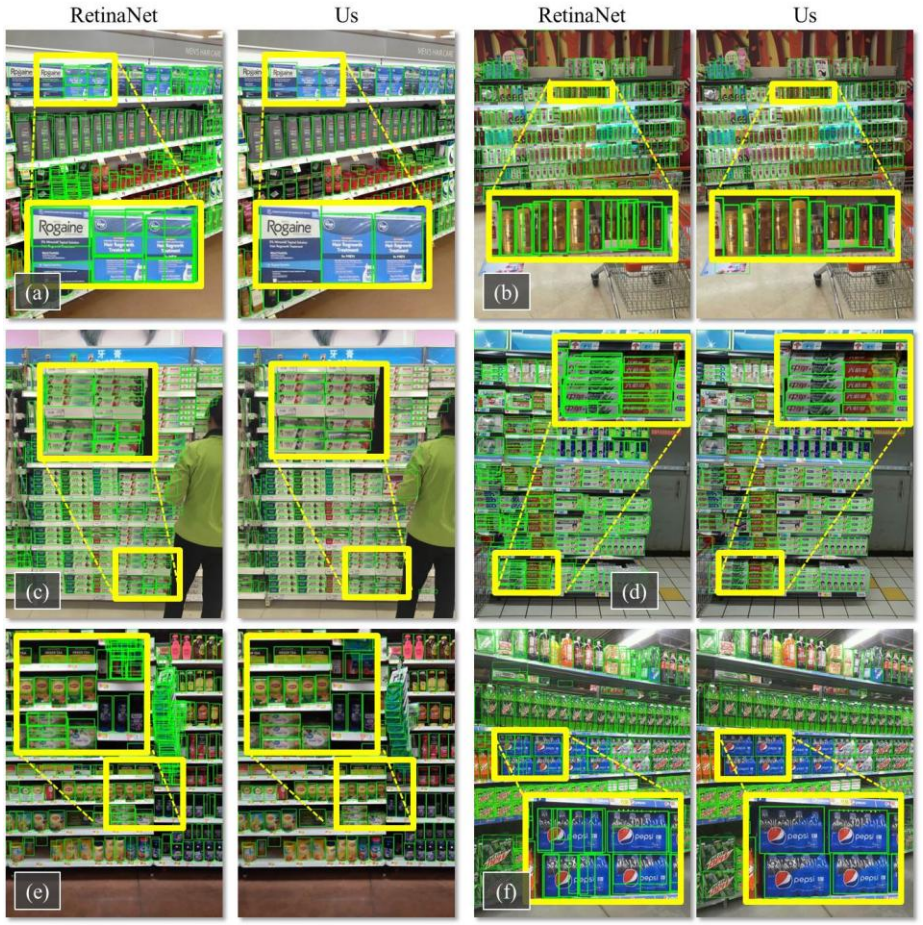
#	Pain point	Who is affected	Consequence	ShelfTwin response	Req.
1	Gaps are found hours late	Shoppers, store	Lost sales for the full gap	Continuous, change-triggered analysis; alert within seconds of confirmed evidence	R1
2	Only 'empty' is detected	Floor staff	Shelf empties during peak hours	Low-stock state, adaptive threshold, time-to-empty estimate	R3
3	False alarms from shoppers, staff, glare	Floor staff	Alerts are ignored	Visibility-weighted evidence across frames; 'cannot see' state	R6
4	No maintained digital planogram	Most Indian stores	Compliance systems cannot start	Layout drafted from shelf tags and stable views; manager approval	R2
5	New, seasonal and look-alike packs	Recognition model	Wrong product, wrong alert	Catalogue retrieval with abstention; add a product with 3-5 photos	R4
6	Stock in backroom, not on shelf	Backroom staff	Avoidable empty shelf	Backroom rack camera; pick location in every task	R5
7	Misplaced and abandoned items	Shoppers, cold chain	Lost sale; spoiled chilled food	Slot alignment; displacement and cold-chain alerts	R4
8	No proof that the shelf was fixed	Store managers	Same gap stays open	Task closes only on camera or photo verification	R5
9	Costly hardware, weak internet	Small and mid-size chains	System cannot be deployed	Reuse CCTV and phones; all inference runs offline in the store	R1
10	No record of availability over time	Category managers	Root causes stay hidden	Per-slot history: time empty, time to fix, repeat offenders	R2

Existing approaches and the gap we address

Approach / work	What it does well	Limitation for this problem
Manual audits	No hardware; staff know the store	Slow, infrequent, error-prone; no time record
Shelf-scanning robots (e.g. Simbe Tally)	Whole-store scans several times a day	High cost; periodic, not continuous
Dense detection: SKU-110K	Finds every pack in crowded shelves	No product identity; single image
Counting: Locount	Localises and counts product groups	Single image; no expected layout
Text matching: Unitail	Reads pack text to match products	Recognition only; no alerts
Planogram compliance (YOLOv8)	YOLOv8 compliance in 7,000+ 7-Eleven stores	Needs a digital planogram; 471 fixed classes
Retail vision platforms (e.g. Everseen)	Store-wide cameras; shelf and loss products	ShelfTwin adds a trusted, verified slot layer

The gap ShelfTwin addresses

- Decisions over time, with explicit uncertainty, instead of per-frame alerts.
- Works from an approved layout when no digital planogram exists.
- Open product catalogue that abstains, and a task loop closed by camera proof.



Dense detection results on real shelves, including a shopper blocking the shelf (panel c).

Detection quality is no longer the bottleneck. Turning detections into trusted actions is.

Proposed solution: ShelfTwin pipeline



(a) Camera frame. People are masked before analysis.



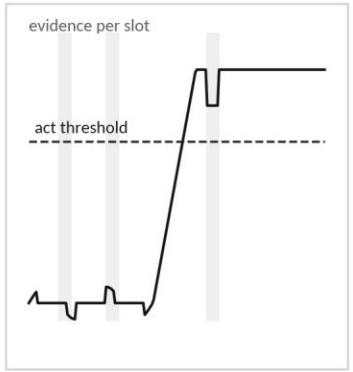
(b) Every pack and gap is detected and boxed.



(c) Detections are mapped to slots of the approved layout.



(d) Packs are matched to the catalogue, or marked unknown.



(e) Evidence builds over frames. Act only above the threshold.



(f) A task with photos, reason and stock location.

1. See and clean

- Frames from shelf cameras, CCTV or phone video.
- Blur, glare and exposure checks reject bad frames.
- A person/cart segmenter builds a visibility mask.

2. Detect and map

- Class-agnostic detector finds packs and gaps.
- Shelf edges give a homography to shelf coordinates.
- Each detection is assigned to a planogram slot.

3. Identify

- Pack crops are embedded (DINOv2, metric-tuned).
- k-nearest neighbours in the store catalogue (FAISS).
- Low margin between top matches gives 'unknown'.

4. Decide

- Per-slot log-odds evidence, weighted by visibility.
- Evidence decays; contradicting views subtract.
- Outputs: ACT, WAIT or CANNOT SEE.

5. Act and verify

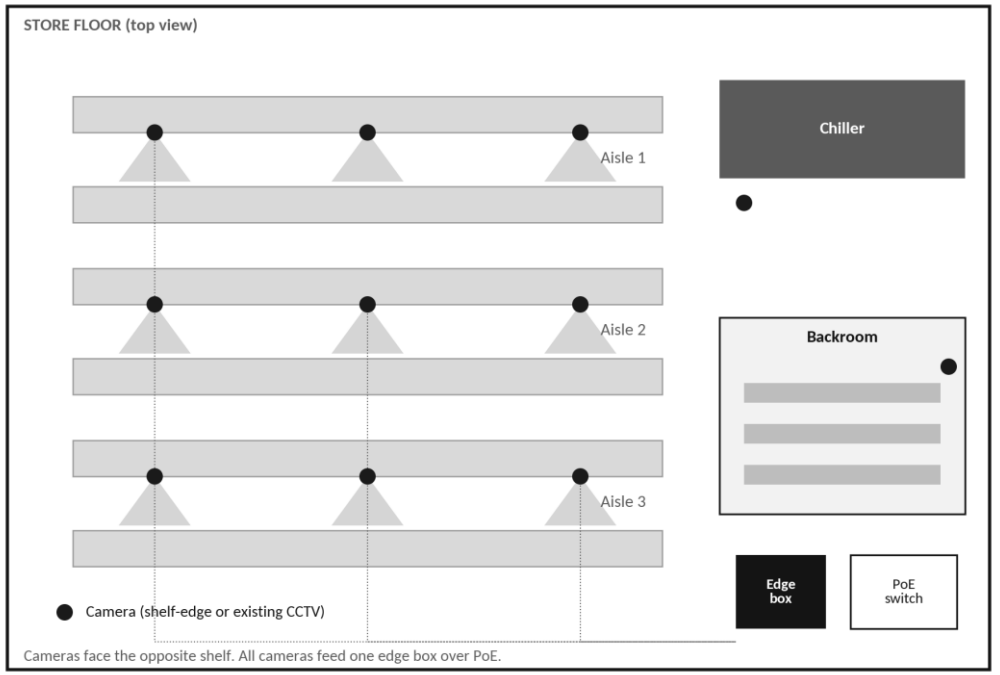
- Ranked task: product, slot, quantity, stock location.
- Shown with before/after crops and the reason.
- Closes only when clean views show the slot full.

The twin: for every slot we store location, expected product (approved layout version), last trusted observation, state (HEALTHY, LOW, EMPTY, MISPLACED, CANNOT SEE), separate detection / identity / visibility / event confidences, and the full history of state changes linked to the frames that caused them. Any alert can be replayed.

System architecture: components and deployment

Component	Technique / model	Runs on	Rate
Frame capture	RTSP/USB readers; hardware video decode (GStreamer)	Edge box	0.5-2 fps per camera
Quality and change gate	Laplacian blur, glare ratio, frame differencing per bay	Edge box	Every frame
Visibility mask	Person/cart instance segmentation (YOLO-seg, small)	Edge box	Changed bays
Pack and gap detector	YOLO (n/s), two classes, 2 tiles per 1080p frame	Edge box	Changed bays
Shelf geometry	Shelf-edge lines, homography to shelf cm, slot assignment	Edge box	On camera move
Product identity	DINOv2 ViT-S/14 + ArcFace head; FAISS k-NN; margin test	Edge box	Changed slots
Pack and tag text	PaddleOCR (mobile); barcode reader for shelf tags	Edge box	Ambiguous only
Planogram matcher	Needleman-Wunsch alignment with position cost	Edge box	Per row update
Temporal state engine	Visibility-weighted log-odds, decay, hysteresis	Edge box	Every observation
Task service	Priority ranking, routing, verification, escalation	Edge + cloud	Event-driven
Twin store and replay	PostgreSQL + object store for masked crops	Edge + cloud	Event-driven
Interfaces	Next.js dashboard; staff web app; Telegram/WhatsApp	Cloud + LAN	Real time

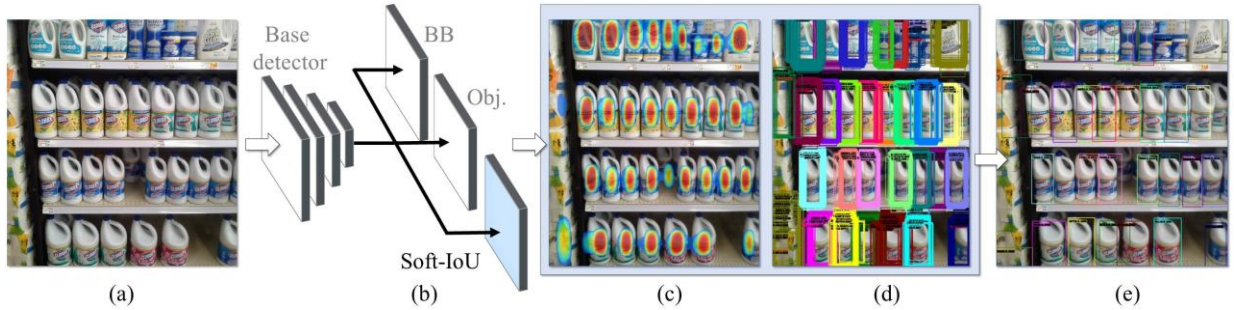
Edge box: NVIDIA Jetson Orin-class (Orin Nano Super, 67 sparse INT8 TOPS), models exported to ONNX/TensorRT (FP16/INT8). Video stays in the store; only events and masked crops are sent out.



Data flow

- Cameras stream to one in-store edge box over PoE.
- The edge box runs the full pipeline and keeps a local task queue.
- Events sync to the cloud twin when the internet is available.
- Staff receive tasks on the store Wi-Fi web app or messaging apps.
- Models, catalogue and rules are versioned and updated in stages.

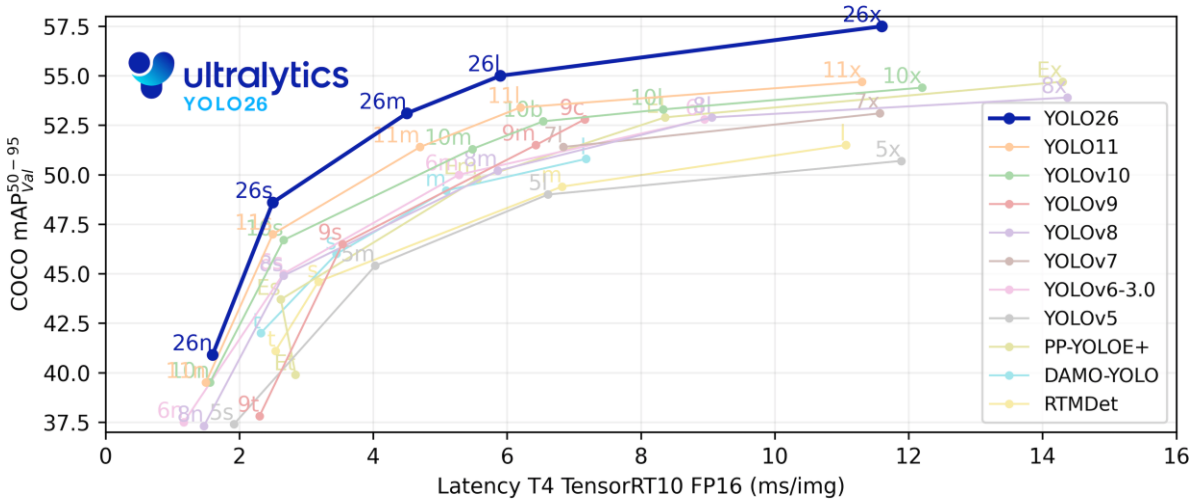
Technique 1: dense pack and gap detection, mapped to shelf slots



Dense detection with a Soft-IoU layer and EM-merger to resolve overlapping boxes.

Detector design

- Class-agnostic 'pack' class plus an explicit 'gap' class (visible shelf back).
- Pre-trained on SKU-110K (11,762 images, 147 objects per image on average).
- Fine-tuned on our own store footage and rendered shelves.
- 1080p frames are split into two 640 px tiles so small packs stay detectable.
- Soft-NMS / box merging handles tightly packed, overlapping facings.



Accuracy vs latency of recent YOLO versions (COCO, T4 GPU, TensorRT). We choose small variants for the edge.

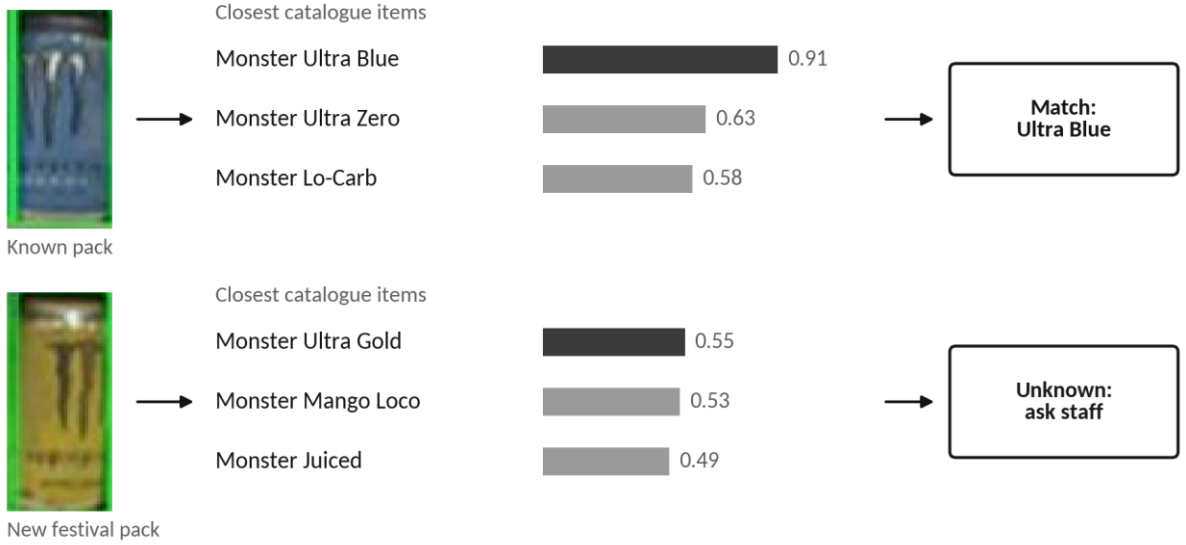
From pixels to slots

- Shelf-edge lines are detected per bay.
- A homography maps image points to shelf coordinates in centimetres.
- Each pack is assigned to a slot by its centre and width.
- Slot coverage = share of the slot face covered by packs.
- Camera drift is detected by feature matching to a reference frame; alerts pause until recalibration.

Small YOLO models reach 45-48 COCO mAP at about 2-3 ms on a T4 GPU, which leaves headroom for many cameras per edge box.

Occlusion: a person/cart mask gives each slot a visibility score (0-1). Hidden slots are never marked empty; their evidence is down-weighted instead.

Technique 2: product identity by catalogue retrieval, with abstention



Real pack crops. Product names and similarity scores are illustrative.

Step	Method	Why
Embed	DINOv2 ViT-S/14 backbone + ArcFace metric head	Strong features for fine-grained packs
Search	FAISS inner-product index over the store catalogue	Millisecond k-NN for thousands of SKUs
Decide	Accept if top score ≥ 0.75 and margin to runner-up ≥ 0.15	Abstain instead of guessing
Tie-break	OCR text overlap: brand, flavour, weight	Separates look-alike variants
Learn	Unknown crops go to an inbox; staff label once	Catalogue grows with use

Why retrieval instead of a fixed classifier

- Stores add and change products every week. A fixed classifier must be retrained each time.
- With retrieval, a new product is one catalogue entry: 3-5 photos and its barcode. No retraining.
- Training uses hard negatives from the same brand, so variants are pushed apart in feature space.
- Both studio pack shots and shelf crops are used, so the catalogue matches real shelf views.

$$s_k = \cos\big(f(x),\, f(g_k)\big), \qquad \text{accept } k^* \iff s_{(1)} \geq 0.75 \;\wedge\; s_{(1)} - s_{(2)} \geq 0.15$$

Identity is kept separate from location

- The expected product of a slot never boosts the match score.
- Location consistency is a separate signal used only by the decision step.
- Result: a misplaced pack is not 'corrected' into the expected product, so displacement stays visible.

Onboarding a new product: the unknown crop appears in the manager inbox; staff scan the barcode and take 3-5 photos; embeddings are computed and pushed to every store index within a minute.

Technique 3: planogram comparison, restock and displacement alerts



Gap is simulated on a real shelf photo (SKU-110K). Matching runs per shelf row, left to right, with a position-aware sequence alignment (Needleman-Wunsch).

Where the expected layout comes from

- Imported: the store's planogram file (CSV/JSON: bay, shelf, x, width, SKU, facings).
- Drafted: shelf-tag barcodes/OCR plus stable observations over the first days.
- Approved: a manager confirms per bay; only approved versions drive alerts.
- Observed, draft and approved layouts stay separate, so a messy shelf is never learned as correct.
- Many simultaneous mismatches in one bay trigger a single 'layout changed?' question, not dozens of alerts.

Row alignment

- Expected slots and observed packs per row are aligned (Needleman-Wunsch).
- Each step is a match, gap (restock), substitution (displacement) or insertion (foreign item).

$$D(i,j) = \min \begin{cases} D(i-1,\, j-1) + \alpha\,|x_i - \hat{x}_j| + \beta\,[\,\mathrm{id}_i \neq \widehat{\mathrm{id}}_j\,] \\ D(i-1,\, j) + \gamma_{\mathrm{gap}} \\ D(i,\, j-1) + \gamma_{\mathrm{ins}} \end{cases}$$

Alert	Trigger	Routed to
Restock: low	Coverage below the slot's threshold	Floor staff, batched per aisle
Restock: empty	No product in the slot	Backroom staff, immediately
Displacement	Product sits in another product's slot	Floor staff
Cold chain	Chilled item seen on an ambient shelf	Floor staff, top priority
Record check	Empty for hours while stock system shows units	Inventory team

Adaptive low-stock threshold

$$\theta_s \;=\; \hat{d}_s\,\left(T_{\mathrm{resp}} + T_{\mathrm{safe}}\right)$$

$\hat{d}_s$: demand rate from billing data, or from the camera's own depletion record. Fast movers warn earlier.

Technique 4: temporal state engine (core contribution)

We treat each slot like a cell in an occupancy grid: every observation adds or removes evidence, weighted by how well the slot was seen.

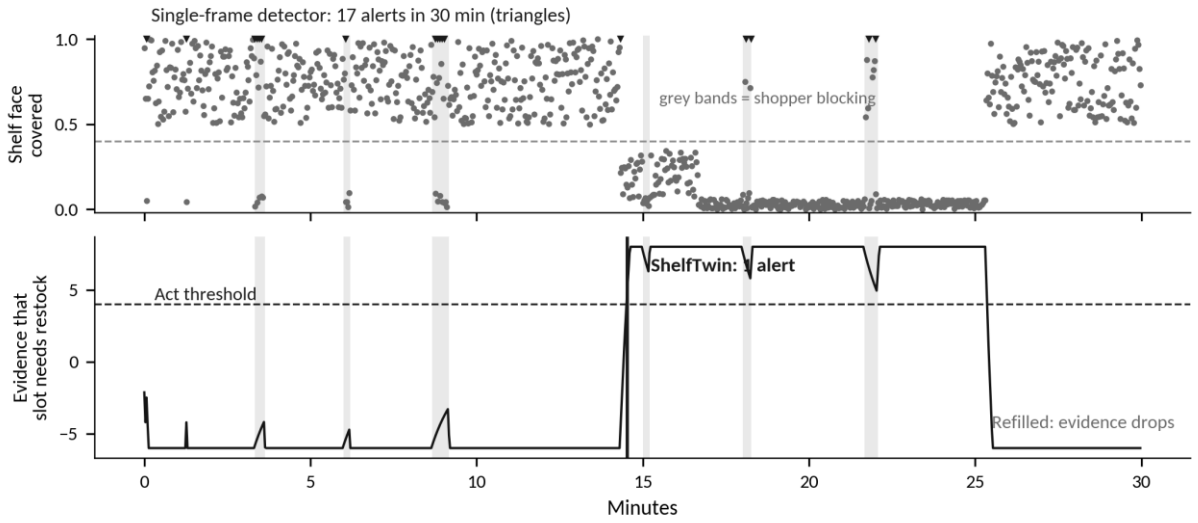
$$L_t = e^{-\lambda \Delta t} L_{t-1} + v_t \, q_t \, \ell(c_t)$$

$$\ell(c) = \begin{cases} +1.6, & c < 0.40 \quad \text{(looks low or empty)} \\ -2.2, & c > 0.45 \quad \text{(looks stocked)} \\ 0, & \text{otherwise} \end{cases}$$

L_t: log-odds that the slot needs restock; $\lambda = 0.02\,\mathrm{s}^{-1}$ (decay)
$v_t, q_t \in [0, 1]$: visibility (person mask) and image quality
c_t: slot coverage; no evidence if $v_t < 0.6$ or $q_t < 0.5$; $-6 \le L_t \le 8$

Rule	Value (starting default)
ACT (open a task)	L ≥ 4.0, about 3 clean confirming views
CLEAR (verify a refill)	L ≤ −1.0 after the task is in progress
Hysteresis	Different enter and exit thresholds stop flicker
CANNOT SEE shown in UI	No clean view for more than 20 s
Evidence cap	−6 ≤ L ≤ 8, so old states can be overturned

- One confident frame can never trigger a task alone.
- Every decision is logged with its inputs, so any alert can be replayed and audited.
- Thresholds are tuned per store on recorded footage, not fixed by hand.



Prototype output for one slot over 30 minutes. Top: per-frame coverage; grey bands show a shopper blocking. Bottom: evidence L(t). The single-frame detector alerts 17 times; the engine alerts once, when stock really runs low, and clears after the refill.

Engine output	Meaning	Staff see
ACT	Evidence is solid	A task with photos and reason
WAIT	Change seen, evidence still weak	Nothing yet
CANNOT SEE	Slot blocked or frame unusable	Last trusted state and its age
CLEAR	Slot verified full again	Task marked verified

Prototype: temporal engine evaluated against baselines

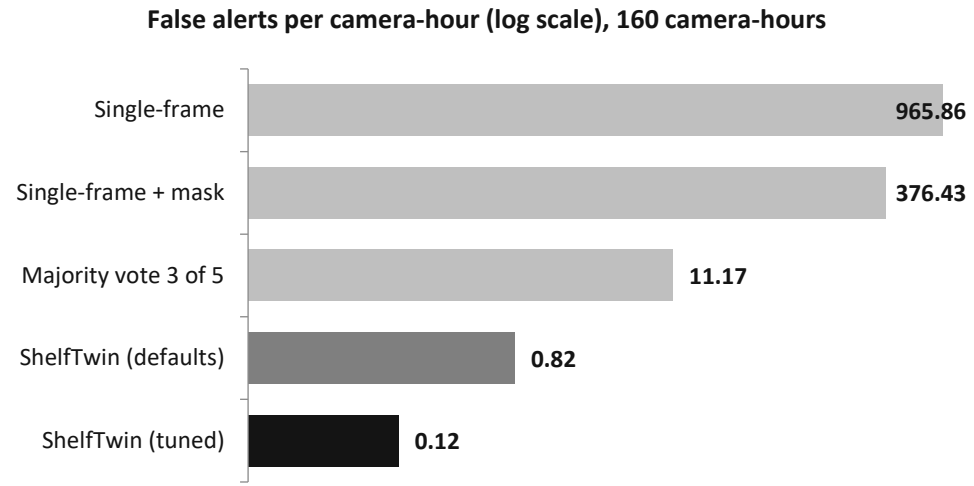
Decision method	False alerts / camera-hour	Precision	Recall	Median delay	95th pct delay
Single-frame	965.86	5.2%	97.5%	0 s	0 s
Single-frame + person mask	376.43	12.3%	99.6%	0 s	2 s
Majority vote 3 of 5 + mask	11.17	45.5%	99.9%	4 s	8 s
ShelfTwin engine (spec defaults)	0.82	91.7%	100%	12 s	18 s
ShelfTwin engine (tuned)	**0.12**	**98.6%**	**100%**	**18 s**	**26 s**

Same simulated observations for every method: 20 runs x 8 hours, 24 slots per camera, 1,452 true restock events. 'Tuned': parameters chosen on separate simulator seeds (100-109), reported on these seeds.

Simulation assumptions

- One frame every 2 s; shoppers take 0.5-6 units per slot per hour (Poisson).
- Shoppers block each slot about 25 times per hour for 2-30 s; the detector sees a 'gap' 70% of that time.
- The person mask misses 5% of blocking events entirely; 1% of clean frames read a false gap (glare).
- Staff refill 10-40 minutes after a slot needs restock, identically for every method.

Scope: these numbers test the decision logic on simulated detector outputs. Trained models on real held-out data: next two slides.



Prototype code (built)

- shelftwin/engine + sim/simulate.py: temporal engine, three baselines and the shelf simulator.
- Full pipeline in code: quality gate, YOLO11 pack+gap detector, geometry, DINOv2+ArcFace identity, planogram alignment, tasks, dashboard.
- 56 automated tests; one Kaggle notebook trains, evaluates and error-analyses every model.

REPOSITORY

github.com/Purv-Kabaria/shelftwin

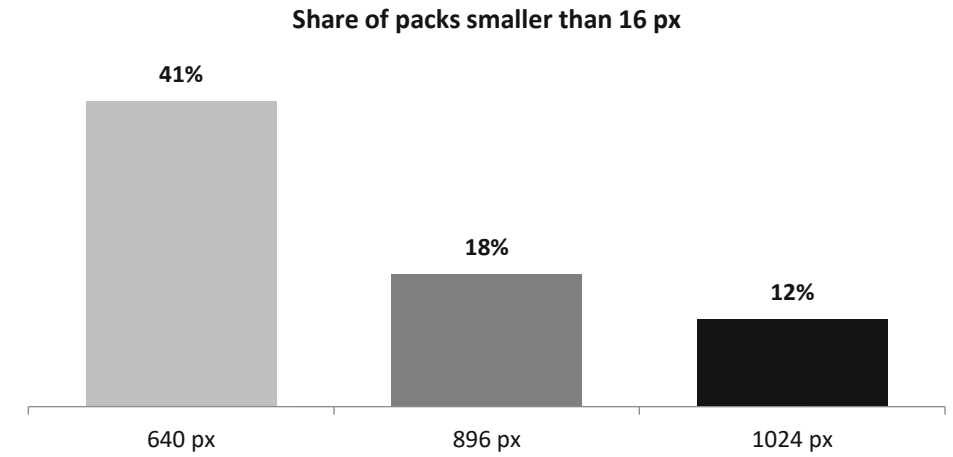
Run: pytest -q · python sim/simulate.py 20. Training logs: kaggle.com/code/purv2k2/shelftwin-training

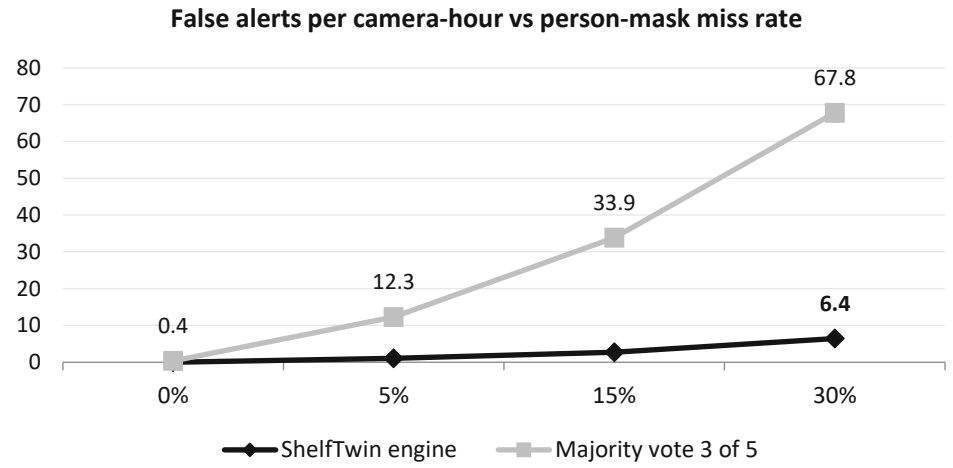
Real data: what the data told us before training

Finding	Measured	Decision it drove
Packs per shelf photo	mean 147, max 718	Keep up to 1000 detections; YOLO11 with NMS (YOLO26 stops at 300)
Packs under 16 px at the training size	640 px: 40.7% 896 px: 17.7% 1024 px: 11.7%	Drop 640 px; compare 896 and 1024 px
Photos with a real shelf gap	88% (5.4 per photo)	Train and test the gap class on real gaps
RP2K product photos	2,233 SKUs, 381k photos	Open-set split by SKU; cap 40 photos per SKU
Engine false alerts (simulation)	19 of 19 involve a missed person mask	Improve the person/cart mask first



Real SKU-110K photos (Kaggle run). Green: annotated packs. Red: gaps found by our shelf-row rule; most are real empty spaces, some are structural (shelf ends).





EDA on all 11,689 SKU-110K photos and 381,303 RP2K photos; engine sweep on 4 x 4 h of simulation.

Trained models: results on held-out real data

95.0%

Pack detection mAP50
mAP50-95 60.2%; 2,920 unseen SKU-110K photos

57.4%

Gap detection AP50
real gaps on unseen photos; recall at 90% precision 7.8%

93.1%

Product identity Recall@1
SKUs never seen in training; accepted precision 94.3%

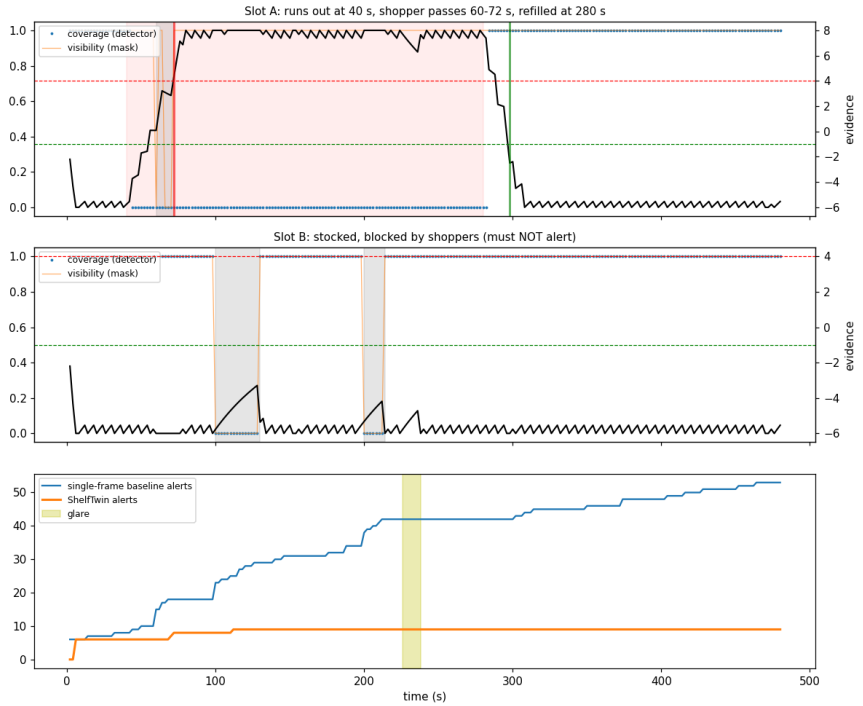
8 vs 53

False alerts: ShelfTwin vs single-frame
real photo, every trained model; stockout caught in 32 s, refill verified

Experiment (subset, short schedule)	Options compared	Winner
Detector: model x resolution x mosaic (mAP50-95)	YOLO11s 1024: 0.567, YOLO11s 896: 0.558, YOLO11s 896 no-mosaic: 0.543, YOLO11n 1024: 0.537	yolo11s @ 1024 px
Gap labels: real only vs + synthetic erasing (gap AP50)	real only: 0.461, mixed: 0.458, shelfback only: 0.471	shelfback only
Identity: fine-tuning depth (Recall@1)	zero-shot 0.862, head only 0.879, last2 0.921, last4 0.952	last4

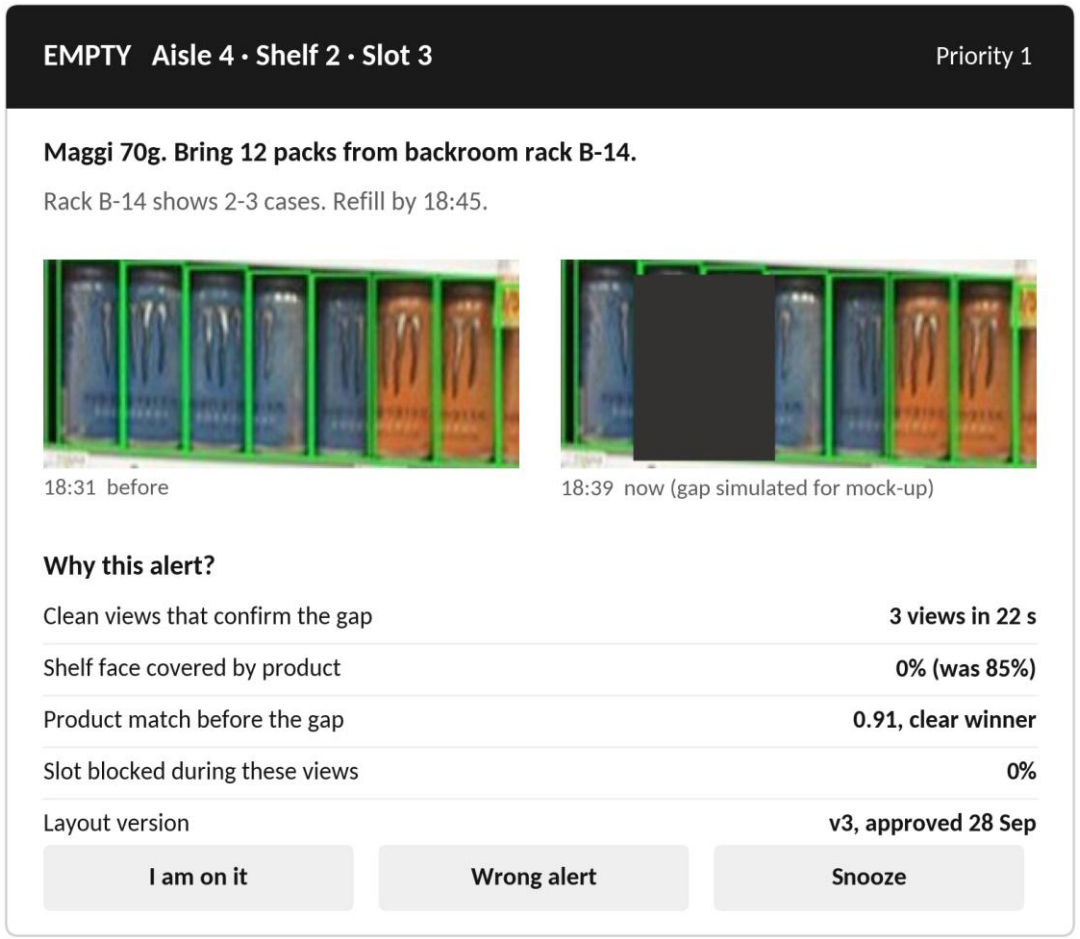
Each experiment ran on a small subset with a short schedule; winners were picked automatically (decision_*.json) and used for the final training on the full data.

- Error analysis per model: misses by pack size and shelf density, counting error, confused SKU pairs, precision vs coverage.
- Most confused pair: 川南黄花下饭菜 vs 川南瓶装四川下饭菜330g (look-alike packs: the system abstains instead of guessing).
- All numbers reproducible from the public Kaggle notebook and the repository.



End-to-end run on a real shelf photo with every trained model: stockout, shoppers, glare, swapped products, refill.

Staff workflow: every alert says what, where, why, and is verified



Design of the staff alert. Real shelf crops; the gap is simulated.

- Channels: web app on store Wi-Fi (works offline), Telegram, WhatsApp.
- Languages: English, Hindi and Gujarati in the final build.
- Feedback: 'Wrong alert' taps become labelled training data.

Task state	Entered when	Leaves when
Open	Engine outputs ACT	Staff tap 'I am on it'
Acknowledged	Staff accept the task	Staff check in at the bay
In progress	Person at bay + app check-in	Staff mark done
Verifying	Staff mark done	Clean views show slot full (CLEAR)
Verified	Camera or photo confirms	Final state; time-to-fix recorded
Reopened	Slot still empty after time limit	Returns to Open with a note

Priority score

$$P_s = \hat{d}_s \, T_{\text{gap}} \, m_s \, p_{\text{lost}} + \beta \, \mathbf{1}[\text{critical}]$$

$\hat{d}_s$: demand rate; T_{gap}: expected gap time; m_s: unit margin
$p_{\text{lost}} = 0.3$–0.5 per category; critical: chilled item misplaced, promotion, top seller

Alert-fatigue controls

- Empty and cold-chain tasks go at once; low-stock tasks batch every 15 minutes per aisle.
- Each team has a cap on open tasks; extra tasks queue nearest-first.
- 'Snooze with reason' (e.g. delivery due at 16:00); unanswered tasks escalate after 10 minutes.

Data strategy: no dependence on organiser-provided data

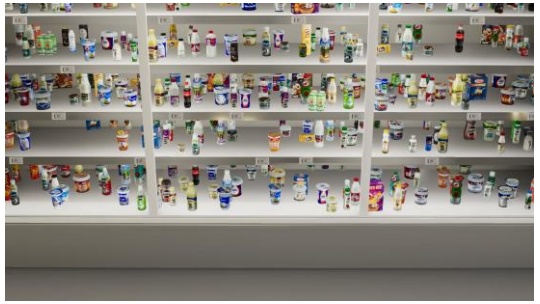
Source	Content	Used for
SKU-110K	11,762 dense shelf images, 1.73M boxes	Pack detector pre-training
Locount	50,394 images, 1.9M instances, 140 classes	Counting and robustness tests
RP2K	2,388 SKUs, about 500k product crops	Metric learning for identity
Retail-786k	786k images, 3,298 product groups	Look-alike training pairs
Unitail-OCR	1,454 products, 30k text regions	Pack text reading
Own store capture	Phone and camera video of local stores, with permission	Fine-tuning, catalogue, test sets
Rendered shelves	Shelves rendered from our pack photos	Rare events: gaps, misplacement, glare

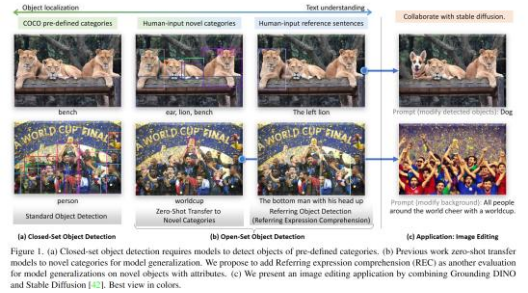
Labelling without hand-drawing every box

- Open-vocabulary teachers (Grounding DINO, SAM 2) propose boxes and masks.
- Labels are kept only where two teachers agree; a sample is checked by hand.
- Staff feedback ('wrong alert', unknown-pack labels) becomes new training data.
- Rendered and edited images are used for training only, never for testing.

Name	#Img.	#Obj./img.	#Cls.	#Cls./img.	Dense.	Idnt.	BB
UCSD (2008) [8]	2000	24.9	1	1	✓	✗	✗
PACAL VOC (2012) [13]	22,531	2.71	20	2	✗	✗	✓
ILSVRC Detection (2014) [12]	516,840	1.12	200	2	✗	✗	✓
COCO (2015) [28]	328,000	7.7	91	3.5	✗	✗	✓
Penguins (2016) [2]	82,000	25	1	1	✓	✗	✗
TRANCOS (2016) [34]	1,244	37.61	1	1	✓	✓	✗
WIDER FACE (2016) [49]	32,203	12	1	1	✗	✗	✓
CityPersons (2017) [51]	5000	6	1	1	✗	✗	✓
PUCPR+ (2017) [22]	125	135	1	1	✓	✓	✓
CARPK (2018) [22]	1448	61	1	1	✓	✓	✓
Open Images V4 (2018) [25]	**1,910,098**	8.4	600	2.3	✗	✓	✓
Our SKU-110K	11,762	**147.4**	**110,712**	**86**	✓	✓	✓

SKU-110K has about 147 objects per image, far denser than COCO (7.7).





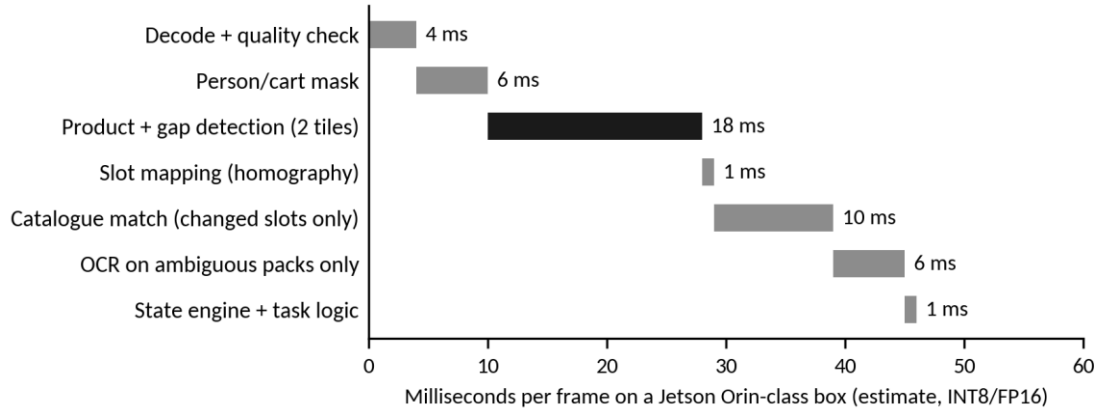
Left: rendered synthetic shelf. Right: open-set detection by Grounding DINO.

Test splits that prevent over-optimism

- Seen sessions; unseen days and lighting; unseen stores.
- Hard cases: look-alikes, multipacks, old vs new packs, half-hidden packs.

Compute, feasibility and scalability

Latency budget: about 46 ms per frame, so one box can serve many cameras



Planning estimates for the final build; to be measured on hardware.

Sizing item	Estimate	Basis
Analysis per frame	about 46 ms	Latency budget above
Box capacity	about 20 frames/s	1000 ms / 46 ms, with headroom
Cameras per box	12-20	0.5-1 frame/s per camera, change-gated
Upload per store	under 200 MB/day	Events + masked crops only
Local storage	about 1 TB SSD	72 h replay buffer + crops

Change gating: heavy models run only on bays whose pixels changed, so quiet aisles cost almost nothing.

Feasibility

- All models are open-source and proven on edge GPUs (YOLO, DINOv2-S, PaddleOCR mobile).
- Works with three camera types; existing CCTV gives bay-level alerts at zero hardware cost.
- No planogram, no inventory system and no internet are required to start.
- Store set-up: mount cameras, walk each aisle once, approve the drafted layout.

Scalability

- Compute scales per store, so head-office cost grows slowly.
- One shared product catalogue; each store loads only its own range.
- Models, catalogue, layouts and rules are versioned separately, rolled out to 1%, 10%, then all stores, with automatic rollback.
- Multi-tenant cloud (row-level security) or private deployment for large chains.
- The same twin extends to pharmacies, dark stores and warehouses.

Camera type	Detail level	Cost
Existing CCTV	Bay-level gaps	None
Shelf-edge camera	Product-level, real time	Per bay
Phone walk-through	Product-level snapshots	None

Costs and return on investment (mid-size Indian supermarket)

Item	Quantity	Unit cost (Rs)	Total (Rs)
Edge GPU box, installed (Jetson Orin class, case, SSD, UPS)	1	45,000-70,000	45,000-70,000
Shelf-edge camera, installed (incl. cabling, mount)	20	6,000-12,000	1.2-2.4 lakh
PoE switch and network work	1	20,000-35,000	20,000-35,000
Existing CCTV reuse	-	0	0
One-time hardware			**about 1.9-3.5 lakh**
Cloud twin, storage, messaging	per year		15,000-30,000
Software subscription and support	per year		1-2 lakh
Maintenance (10-15% of hardware)	per year		20,000-50,000

Planning estimates; to be confirmed with supplier quotes. Phone-only and CCTV-only modes need no new hardware.

Worked example

	Value
Store revenue	Rs 10 crore / year
Lost to out-of-stocks (3-4%)	Rs 30-40 lakh / year
Recovered (assume 25-50%)	Rs 7.5-20 lakh / year
Gross margin 20-25%	Rs 1.5-5 lakh profit / year
Plus staff hours saved	Audits replaced by tasks
Payback	**about 6-24 months**

Recovery share is an assumption. Vendor case studies report on-shelf availability gains of 25-60%, which a pilot must confirm for Indian stores.

Cost levers

- Shelf cameras only on the 20-30% of bays that carry most sales; CCTV or phones elsewhere.
- Change gating lets one box serve more cameras.
- Hardware can be leased inside the subscription to remove upfront cost.

Other value

- Accurate shelf data improves ordering.
- Fewer price and placement errors.
- Brand compliance data with consent.

Privacy, security and limitations

Privacy by design

- Continuous video never leaves the store.
- People, hands, phones and badges are masked in every saved crop.
- No face recognition, no re-identification, no tracking of people.
- Crops kept 30 days by default; replay frames 72 hours.
- Staff metrics are team-level, not individual.
- Each deployment is reviewed under India's DPDP Act 2023 and Rules 2025.

Security

- Edge box makes outbound connections only; no open ports.
- Mutual TLS between store and cloud; per-device certificates.
- Signed model and software updates; secure boot.
- Encrypted disks; role-based access; every crop view logged.
- Camera network isolated on its own VLAN.

What version 1 does not claim

- Exact unit counts or store-wide inventory.
- Automatic planogram changes or ordering.
- Perfect separation of near-identical packs (it abstains).
- Product-level identity from wide ceiling CCTV.
- Camera depletion is not called 'sales' without billing data.

Data	Leaves the store?	Retention	Who can see it
Live video	No	Seconds (memory only)	Nobody
Replay frames (people masked)	No	72 hours	Store admin, on request
Slot crops (people masked)	Yes, with events	30 days	Store and HQ roles
Event and task records	Yes	Configurable	Store and HQ roles
Model and system telemetry	Yes	90 days	Operations team

Prototype status and the planned final build

Module	In the prototype now	Planned for the final build
Temporal state engine	**Built and tested; tuned on held-out simulator seeds**	Tuned on real footage; per-store thresholds
Baselines and simulator	**Built, with false-alert error analysis**	Extended with recorded real detections
Pack and gap detector	**Built: YOLO11 trained on SKU-110K (Kaggle); gap labels from shelf geometry**	Fine-tune on Indian-store footage; TensorRT on edge
Visibility mask	**Built with a pretrained person segmenter**	Fine-tune for carts and baskets
Product identity	**Built: DINOv2 + ArcFace trained on RP2K; FAISS; abstention**	Indian pack catalogue (3-5 photos per SKU); OCR tie-break
Planogram matcher	**Built and tested (row alignment, layout guard)**	Layout drafting from shelf tags; approval flow
Alerts and tasks	**Built: priority, batching, escalation, camera verification; Telegram; en/hi/gu**	Staff mobile app, WhatsApp channel
Twin dashboard	**Built: live slot view, tasks, alert replay**	Store-wide, multi-camera view
Backroom routing	**Rack location carried in every task**	Rack camera; pick location from stock
Time-to-empty	**Built: Gamma-Poisson estimate with a range**	Use billing data when connected

Final build: what it will show

- A shelf with 60-80 real Indian SKUs and 3-4 cameras.
- Shoppers block views; the baseline raises false alerts while ShelfTwin waits.
- A real low-stock alert in Hindi names the backroom rack.
- A refill is verified by the camera and the task closes.
- An unseen pack is flagged 'unknown' and added in a minute.

Measures we will report

- Alert precision and recall per episode.
- False alerts per camera-hour.
- Detection and notification delay.
- Correct 'unknown' rate on new packs.
- Cameras per box at full speed.

Main risks and mitigation

- Look-alike packs: abstain and ask staff.
- Low-resolution CCTV: bay-level alerts only.
- Camera moved: drift check pauses alerts.

References

[1] T. W. Gruen, D. S. Corsten, S. Bharadwaj. Retail Out-of-Stocks: A Worldwide Examination of Extent, Causes and Consumer Responses. Grocery Manufacturers of America, 2002.

[2] D. Corsten, T. Gruen. Desperately seeking shelf availability: an examination of the extent, the causes, and the efforts to address retail out-of-stocks. IJRDM 31(12), 2003.

[3] E. Goldman, R. Herzig, A. Eisenschtat, J. Goldberger, T. Hassner. Precise Detection in Densely Packed Scenes (SKU-110K). CVPR 2019.

[4] Y. Cai et al. Rethinking Object Detection in Retail Stores (Locount). AAAI 2021.

[5] F. Chen et al. Unitail: Detecting, Reading, and Matching in Retail Scene. ECCV 2022.

[6] J. Peng, C. Xiao, Y. Li. RP2K: A Large-Scale Retail Product Dataset for Fine-Grained Image Classification. arXiv:2006.12634, 2020.

[7] B. Lamm, J. Keuper. Retail-786k: a Large-Scale Dataset for Visual Entity Matching. arXiv:2309.17164, 2023.

[8] T.-Y. Ou, A. Ponce, C. Lee, A. Wu. Real-time retail planogram compliance application using computer vision and virtual shelves. Scientific Reports 15, 43898, 2025.

[9] M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. TMLR, 2024.

[10] G. Jocher et al. Ultralytics YOLO (YOLOv8, YOLO11, YOLO26). Software and benchmarks, github.com/ultralytics/ultralytics.

[11] J. Deng, J. Guo, N. Xue, S. Zafeiriou. ArcFace: Additive Angular Margin Loss for Deep Face Recognition. CVPR 2019.

[12] J. Johnson, M. Douze, H. Jegou. Billion-scale similarity search with GPUs (FAISS). IEEE Trans. Big Data, 2019.

[13] S. B. Needleman, C. D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. J. Mol. Biol. 48(3), 1970.

[14] H. Moravec, A. Elfes. High resolution maps from wide angle sonar. IEEE ICRA, 1985.

[15] S. Liu et al. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. ECCV 2024.

[16] Y. Du et al. PP-OCR: A Practical Ultra Lightweight OCR System. arXiv:2009.09941, 2020 (PaddleOCR).

[17] Neurolabs. CPGDet-129: Synthetic Dataset for Object Detection in Retail. github.com/neurolaboratories/reshelf-detection, 2022.

[18] M. Denninger et al. BlenderProc2: A Procedural Pipeline for Photorealistic Rendering. JOSS 8(82), 2023.

[19] NVIDIA. Jetson Orin Nano Developer Kit Gets a Super Boost. NVIDIA Technical Blog, December 2024.

[20] N. Ravi et al. SAM 2: Segment Anything in Images and Videos. arXiv:2408.00714, 2024.

[21] Simbe Robotics. Wakefern Food Corp. customer story (on-shelf availability results). simberobotics.com, accessed 2026.

[22] Everseen. Evershelf product page; Everseen expanded product suite incl. Everstock (press release, Jan 2023).

[23] Ministry of Electronics and IT, Government of India. Digital Personal Data Protection Rules, 2025 (notified November 2025).